

NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE & AI

MIDTERM EXAMINATION (PAPER-BASED)

Course: Data Analysis with Python (DSAI1005)

Scope: NumPy (20%), Pandas (20%), Viz (20%), Code (40%)

Difficulty: 60% Easy/Basic (Section I) + 40% Medium (Section II)

Duration: 60 minutes — **Exam Code:** 301-EN

(No personal laptops, textbooks, or unauthorized notes allowed)

Full Name: **Student ID:** **Exam Code:** 301-EN

Class / Cohort: **Exam Room:** **Seat No:**

MULTIPLE-CHOICE ANSWER SHEET *(Write letters A, B, C, or D in the corresponding boxes):*

Question	1	2	3	4	5	6	7	8	9	10	11	12
Answer												

SECTION I: MULTIPLE-CHOICE QUESTIONS (6.0 pts — 0.5 pt each)

(Easy & Basic Foundational Questions — 60% of Total Exam Score)

1. What array is produced by the vectorized arithmetic expression below?

```
import numpy as np
a = np.linspace(0, 10, 5)
b = np.arange(1, 6)
res = a + b * 2
```

- A. [2., 6.5, 11., 15.5, 20.] C. [2, 7, 11, 16, 20]
B. [2., 4.5, 9., 13.5, 18.] D. [0., 4.5, 11., 15.5, 22.]

2. In NumPy, basic slicing creates an array *view* rather than an independent memory copy:

```
m = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
v = m[:, -1, 1]
v[0] = 999
print(m[2, 1], v[2])
```

What values are printed to the console?

- A. 80 20 B. 999 20 C. 80 999 D. 999 80

3. Array *A* has shape (4, 1, 6) and array *B* has shape (3, 1). Under NumPy broadcasting rules, what is the shape of the evaluated array from the expression `C = A * B + np.zeros(6)`?

- A. (4, 3, 6) B. (4, 1, 6) C. (3, 4, 6) D. Raises
ValueError

4. Consider the following conditional filtering and axis reduction snippet:

```
x = np.array([[12, 25, 8], [30, 15, 22]])
filtered = np.where((x % 2 == 0) & (x > 10), x * 2, 0)
print(np.sum(filtered, axis=0))
```

What output array is displayed?

- A. [84, 0, 44] B. [24, 60] C. [84, 50, 44] D. [24, 0, 44]
5. When performing arithmetic addition between two Pandas Series S_1 and S_2 with non-identical string index labels:

```
import pandas as pd
s1 = pd.Series([10, 20, 30], index=['Hanoi', 'HCMC', 'Da Nang'])
s2 = pd.Series([5, 15, 25], index=['HCMC', 'Can Tho', 'Hanoi'])
res = s1.add(s2, fill_value=0)
```

What are the resulting values of `res['Da Nang']` and `res['Can Tho']`?

- A. Both evaluate to NaN. C. An `IndexMismatchError` is raised.
 B. 30.0 and 15.0 respectively. D. 0.0 and 0.0 respectively.
6. Consider the sales performance DataFrame `df`:

```
df = pd.DataFrame({
    'Revenue': [150, 280, 310, 450],
    'Cost': [100, 190, 200, 300]
}, index=['Q1', 'Q2', 'Q3', 'Q4'])
loc_slice = df.loc['Q2':'Q4', 'Revenue']
iloc_slice = df.iloc[1:3, 0]
```

How many elements are contained in `loc_slice` and `iloc_slice` respectively?

- A. `loc_slice`: 3 elements; `iloc_slice`: 2 elements C. `loc_slice`: 3 elements; `iloc_slice`: 3 elements
 B. `loc_slice`: 2 elements; `iloc_slice`: 2 elements D. `loc_slice`: 2 elements; `iloc_slice`: 3 elements
7. Given the multi-level aggregation pipeline on retail transactions:

```
orders = pd.DataFrame({
    'Region': ['North', 'South', 'North', 'South', 'North'],
    'Category': ['Food', 'Food', 'Tech', 'Tech', 'Food'],
    'Sales': [100, 150, 300, 250, 200]
})
summary = orders.groupby(['Region', 'Category'])['Sales'].agg(['sum', 'count'])
print(summary.loc[('North', 'Food'), 'sum'])
```

What value is printed to the console?

- A. 100 B. 300 C. 2 D. 450
8. You execute a Left Merge between table A ($N = 500$ rows) and table B ($M = 80$ rows) on key 'store_id':

```
merged = pd.merge(A, B, on='store_id', how='left')
```

Assuming 'store_id' is unique in table B, but 30 rows in table A reference a `store_id` that does not exist in table B. Which statement is true regarding `merged`?

- A. `merged` contains exactly 470 rows, dropping the 30 unmatched entries.
 B. `merged` contains exactly 500 rows, and columns originating from table B will have NaN for those 30 rows.
 C. `merged` contains exactly 80 rows, matching only the intersection.
 D. A `KeyError` is thrown because all foreign keys must exist in the right table.

9. In Matplotlib's Object-Oriented interface, consider the canvas creation:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
```

Which statement correctly accesses and configures the subplot located in the **first row, second column**?

- A. `axes[0, 1].set_title("Branch Q2")` D. `plt.subplot(1, 2).set_title("Branch Q2")`
B. `axes[1, 2].set_title("Branch Q2")`
C. `axes[1, 0].title = "Branch Q2"`
10. In exploratory data analysis, when analyzing continuous financial transaction amounts containing substantial positive skewness and extreme outliers, which visualization tool is most appropriate for observing the median, interquartile range (*IQR*), and whisker thresholds simultaneously?
- A. `sns.barplot()` B. `sns.boxplot()` C. `sns.countplot()` D. `sns.lineplot()`
11. Consider the following Seaborn relational plotting statement:

```
import seaborn as sns
sns.scatterplot(data=df, x='Marketing_Spend', y='Revenue', hue='Channel', style='Tier')
```

What role do the parameters `hue='Channel'` and `style='Tier'` play in the resulting visualization?

- A. `hue` maps channel categories to distinct point colors; `style` maps tier categories to distinct marker shapes (glyphs).
B. `hue` adjusts point opacity; `style` sets the background CSS layout.
C. `hue` partitions data into separate subplots; `style` specifies the font style.
D. Both parameters are synonymous and map data to line width.
12. Given the Pearson correlation matrix computation and visualization:

```
corr_matrix = df[['Age', 'Income', 'Score', 'Tenure']].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.tight_layout()
```

What is the primary function of `annot=True` and `plt.tight_layout()` respectively?

- A. `annot=True` adds interactive tooltips; `plt.tight_layout()` compresses the image resolution.
B. `annot=True` writes numeric correlation coefficients inside each cell; `plt.tight_layout()` adjusts subplot margins to prevent label clipping.
C. `annot=True` filters out correlations near zero; `plt.tight_layout()` converts the plot to black and white.
D. `annot=True` prints p-values; `plt.tight_layout()` locks axes aspect ratios to 1:1.

SECTION II: APPLIED CODING & SHORT-ANSWER (4.0 pts)

(Medium-Level Integrated Pipeline Problems — 40% of Total Exam Score)

Question 1: Vectorized Computations & Tabular Aggregation with NumPy & Pandas (2.0 pts)

Scenario: An enterprise processes transactional batches. You are provided with a raw 2D array `tx_data` (5×4), with column metrics: 0: `unit_price`, 1: `quantity`, 2: `discount_rate`, 3: `shipping_cost`:

```
import numpy as np
import pandas as pd

tx_data = np.array([
    [50000.0, 3.0, 0.10, 15000.0], [120000.0, 1.0, 0.00, 20000.0],
    [45000.0, 4.0, 0.05, 10000.0], [300000.0, 2.0, 0.20, 25000.0],
    [80000.0, 5.0, 0.15, 15000.0]
])
```

Tasks (write exact and concise Python statements):

- (0.4 pts)** Write a vectorized expression to compute 1D array `net_revenue` across all orders: `net_revenue = (unit_price × quantity × (1 − discount_rate)) − shipping_cost`.
- (0.4 pts)** Using `np.where`, create 1D string array `order_tier` tagging an order as "Wholesale" if `net_revenue ≥ 250 000` **or** `quantity ≥ 4`, otherwise "Retail".
- (0.4 pts)** Given `orders_df` (cols: 'order_id', 'cust_id', 'net_rev', 'tier') and `cust_df` (cols: 'cust_id', 'city', 'status'). Merge `orders_df` with `cust_df` on 'cust_id' retaining all orders (Left Merge).
- (0.4 pts)** On merged, chain `.groupby()` and `.agg()` by ['city', 'tier'] to compute the **sum** and **count** of 'net_rev'.
- (0.4 pts)** Write a statement to sort `summary` by total revenue (the sum of `net_rev`) in **descending** order.

Work Area — Question 1:

```
a) net_revenue = .....

b) order_tier = .....

c) merged = orders_df.merge(.....)

d) summary = merged.groupby(.....)
    .....

e) final_sorted = summary.sort_values(.....)
```

Question 2: Multi-Panel Visual Analytics & Outlier Detection with Pandas & Seaborn (1.5 pts)

Scenario: You are building an executive multi-panel analytics dashboard to inspect sales distributions across payment channels and monitor fraudulent risk anomalies. You are provided with a pre-loaded DataFrame `df_sales`:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Schema of df_sales:
# 'order_date': string formatted ('2026-09-01', ...)
# 'channel': string ('Online', 'Store', 'Partner')
# 'payment': string ('Credit Card', 'Cash', 'Bank Transfer')
# 'amount': float (order value in VND)
# 'is_fraud': int (1 for suspicious/fraudulent, 0 for legitimate)
```

Tasks (write correct Python code):

- (0.5 pts)** Write a Matplotlib Object-Oriented statement to initialize a canvas containing a 1×2 grid (1 row, 2 columns of subplots) with figure dimensions of 14×5 inches, unpacking into figure object `fig` and axes array `axes`.
- (1.0 pts)** Write a complete **Seaborn** statement to render a grouped Boxplot on `axes[0]` displaying transaction amounts (`y= 'amount'`) across payment methods (`x= 'payment'`), stratified by fraud status (`hue= 'is_fraud'`), styled with palette `'Set2'`, targeted specifically to axis `axes[0]`.

After rendering, the left subplot is labeled and formatted with the following commands:

```
axes[0].set_title("Transaction Amount Distribution by Payment Method", fontsize=12)
axes[0].set_xlabel("Payment Method", fontsize=10)
axes[0].set_ylabel("Order Value (VND)", fontsize=10)
```

Work Area — Question 2:

a) Matplotlib OO canvas initialization (0.5 pts):

`fig, axes = .....`

b) Complete Seaborn grouped boxplot call (1.0 pts):

```
sns.boxplot(
    .....
    .....
    .....
    .....
)
```

Primary analytical purpose of setting `hue='is_fraud'`:

Question 3: Time-Series Revenue Trajectory & Executive Decision-Making (0.5 pts)

Scenario: On the second subplot (`axes[1]`), order dates are indexed as datetimes to track weekly sales momentum and forecast quarterly turnover:

```
df_sales['order_date'] = pd.to_datetime(df_sales['order_date'])
df_sales = df_sales.set_index('order_date')
weekly_sales = df_sales['amount'].resample('W').sum()
```

Tasks:

- a) (0.3 pts) Write a Pandas statement using `weekly_sales` to compute a **4-week rolling moving average** named `rolling_trend` to smooth weekly variance. Then write the standard Matplotlib call to automatically adjust canvas padding, preventing label clipping across subplots.
- b) (0.2 pts) Formulate **two concise data-driven takeaways / policies**: (1) how an analyst spots fraudulent transaction outliers on `axes[0]` using *IQR* whiskers, and (2) why a 4-week moving average on `axes[1]` is analytically superior to raw weekly spikes for revenue forecasting.

Both actual weekly sales and the smoothed rolling trend are plotted on `axes[1]`:

```
axes[1].plot(weekly_sales.index, weekly_sales.values, label='Weekly Actual', color='steelblue', alpha=0.6)
axes[1].plot(weekly_sales.index, rolling_trend.values, label='4-Week Trend', color='crimson', linewidth=2)
axes[1].set_title("Weekly Revenue Trajectory & 4-Week Moving Average", fontsize=12)
axes[1].legend(); axes[1].grid(True, linestyle='--', alpha=0.5)
```

Work Area — Question 3:**a) Pandas 4-week rolling average & canvas padding (0.3 pts):**

`rolling_trend = weekly_sales.....`

Command to prevent label clipping:

b) Executive Insights & Takeaways (0.2 pts):

1. Outlier Identification via Boxplot Whiskers:

2. Forecasting Advantage of 4-Week Rolling Average:

————— **END OF EXAM** —————

(Proctors will not answer questions regarding exam content. Hand in all exam sheets upon completion.)